

P8X Programmer's Guide

Instruction set rev 1 - generated from the microcode source (genucode.py); opcode values, byte and cycle counts are extracted from the same tables that build the EPROM images, so this document cannot drift from the hardware.

Programming model

A, B - 8-bit ALU operand registers; results land in A. **P0-P3** - four 16-bit pointer registers; the address bus is always driven by one of them. **P0** is the program counter. **P3** is the stack pointer (empty-descending: push writes then decrements; initialise it early - see note 3). P1 and P2 are general pointers used by the (Pn) addressing modes. **FLAGS** - C, Z, N, V, latched only by instructions marked in the Flags column.

Memory map: \$0000-\$7FFF EEPROM | \$8000-\$FEFF RAM | \$FF00-\$FFFF I/O: \$FF00 switches (r), \$FF02 LEDs (w), \$FF04 ACIA status (bit0 RX ready, bit1 TX ready), \$FF05 ACIA data.

Reset: P0 is forced to \$0000; execution begins there. All other registers (including P3) are undefined on real hardware.

Instruction set

Op	Mnemonic	Bytes	Cycles	Flags	Description
System					
\$00	NOP	1	2	-	No operation.
\$01	HLT	1	2	-	Halt the clock. Resume only by reset (or emulator exit).
Load / store					
\$10	LDA #imm	2	2	-	A := immediate byte.
\$11	LDB #imm	2	2	-	B := immediate byte.
\$15	LDA (P1)+	1	2	-	A := memory at P1, then P1 := P1 + 1.
\$16	LDA (P2)+	1	2	-	A := memory at P2, then P2 := P2 + 1.
\$17	LDA (P3)+	1	2	-	A := memory at P3, then P3 := P3 + 1.
\$19	STA (P1)+	1	2	-	Memory at P1 := A, then P1 := P1 + 1.
\$1A	STA (P2)+	1	2	-	Memory at P2 := A, then P2 := P2 + 1.
\$1B	STA (P3)+	1	2	-	Memory at P3 := A, then P3 := P3 + 1.
\$1D	STA (P1)	1	2	-	Memory at P1 := A.
\$1E	STA (P2)	1	2	-	Memory at P2 := A.
\$1F	STA (P3)	1	2	-	Memory at P3 := A.
ALU (operands A,B; result to A unless noted)					
\$20	ADD	1	2	C Z N	A := A + B.
\$21	SUB	1	2	C Z N	A := A - B.
\$22	AND	1	2	C Z N	A := A AND B.
\$23	OR	1	2	C Z N	A := A OR B.
\$24	XOR	1	2	C Z N	A := A XOR B.
\$25	CMP	1	2	C Z N	Flags from A - B; A unchanged.
\$26	INC	1	2	C Z N	A := A + 1. (B not used.)
\$27	DEC	1	2	C Z N	A := A - 1. (B not used.)
\$28	SHL	1	2	C Z N	A := A << 1, 0 into bit 0. (1)
\$29	SHR	1	2	C Z N	A := A >> 1, 0 into bit 7. (1)
Pointer registers					
\$31	LPL1 #imm	2	3	-	Low byte of P1 := immediate.
\$32	LPL2 #imm	2	3	-	Low byte of P2 := immediate.
\$33	LPL3 #imm	2	3	-	Low byte of P3 := immediate.

Op	Mnemonic	Bytes	Cycles	Flags	Description
\$35	LPH1 #imm	2	3	-	High byte of P1 := immediate.
\$36	LPH2 #imm	2	3	-	High byte of P2 := immediate.
\$37	LPH3 #imm	2	3	-	High byte of P3 := immediate.
Control flow					
\$40	JMP addr	3	5	-	P0 (PC) := addr.
\$41	JSR (P1)	1	9	-	Push return address (high byte first) onto P3 stack, then P0 := P1. Target must already be in P1.
\$42	RTS	1	7	-	Pop return address from P3 stack into P0.
\$48	BZ addr	3	5	-	Branch to addr if Z=1.
\$49	BNZ addr	3	5	-	Branch to addr if Z=0.
\$4A	BCP addr	3	5	-	Branch if C=1, i.e. the RAW 74181 Cn+4 pin is high. Pin high means NO carry out - see note (2).
-	LDPn #imm16	4	6	-	Assembler pseudo-op: LPLn + LPHn pair. Pn := imm16.

Notes

(1) Shifts: the shift-in bit is wired to the microcode CIN line and is 0 for SHL/SHR. The C flag latches the 74181 carry chain, not the shifted-out bit, in rev A.

(2) C flag polarity: the flag register latches the RAW 74181 Cn+4 pin, which is an ACTIVE-LOW carry: after ADD, C=0 means a carry occurred; after SUB/CMP, C=0 means no borrow (A >= B). BCP branches on the pin being HIGH. This matches the rev A ALU card exactly; a rev B inverter (or embracing the convention) is an open BACKLOG item. The emulator reproduces it faithfully.

(3) Stack: JSR pushes the return address high byte then low byte, decrementing after each write (empty-descending). RTS increments then reads. Software must initialise P3 (e.g. LDP3 #\$FEFF) before the first JSR. **(4) V flag:** reads 0 in rev A. **(5) JSR addressing:** only (P1) - load the target into P1 first; an absolute-JSR opcode is a natural future addition.

Assembler quick reference (p8xasm.py)

```
label:  MNEMONIC operand          ; comment
operands:  #expr (immediate) | (Pn) / (Pn)+ | expr (16-bit address)
exprs:    $1F 0x1F 31 'c' symbol with +/-, <expr = low byte, >expr = high
directives: .org e .byte e,... .word e,... .ascii "s" .asciiz "s"
            .fill n[,v] NAME = expr .equ NAME, expr
pseudo-op: LDPn #imm16            ; expands to LPLn #<imm, LPHn #>imm
usage:    python3 p8xasm.py prog.asm -o eeprom.bin [-l prog.lst]
```

Example: print a string

```
ACIA_D = $FF05
.org 0
LDP2 #ACIA_D      ; P2 -> ACIA data register
LDP1 #msg         ; P1 -> string
LDB #0
loop: LDA (P1)+    ; fetch byte, advance
      OR          ; A := A|0 - sets Z on the terminator
      BZ done
      STA (P2)    ; transmit
      JMP loop
done: HLT
msg:  .asciiz "P8X lives!\r\n"
```